Introduction

Name: Mehak Faheem

Roll no: 23F-AI-47

Section: A-1

Department: BSAI

Date: 16th April 2026

Submitted to: Ms.Raheela Shah

Task 1: Automated Backup & Cleanup System

Operating Systems OEL-1

1. Objective

- Task 1: Automated Backup & Cleanup System Objective. Design a script-based solution to manage files/directories with backup and cleanup logic. Your system's /project directory has multiple subdirectories created by different users. Over time it gets cluttered with old .log, .tmp, and large .zip files. Build a shell script that creates a backup_YYYY-MM-DD folder inside /backup. Moves all files older than 30 days from /project into it, preserving the subdirectory structure. Deletes .tmp files older than 7 days directly without backup. Generates a report.txt listing: files moved, files deleted, total space cleared, and any permission errors faced. Logs all actions with timestamps to cleanup.log. Open-ended part: Decide how to handle name conflicts, permissions for other users' files, and what to do if /backup runs out of space. Test with your own sample directory tree. find, mv, cp, rm, date, mkdir -p, du, chmod, >> logging, conditional statements.

2. Environment & Directory Setup

2.1 Sample Directory Tree Creation

The following commands were run to create the test environment:

```
mkdir -p /home/mehakproject/alice/reports
mkdir -p /home/mehak/project/bob/logs
mkdir -p /home/mehak/project/charlie/temp
mkdir -p /home/mehak/project/shared/archives
mkdir -p /home/mehak/backup
```

Files were then created with different ages using touch -t to simulate a realistic mix of old and recent files:

```
# Old .log and .zip files (> 30 days old)
touch -t 202502010000 project/shared/archives/backup_old.zip
touch -t 202502150000 project/charlie/temp/cache.zip
touch -t 202502200000 project/alice/reports/q1_summary.log
touch -t 202502250000 project/alice/reports/q2_data.log
touch -t 202503010000 project/bob/logs/server.log

# Old .tmp files (> 7 days old)
touch -t 202604010000 project/alice/reports/draft.tmp
touch -t 202604030000 project/bob/logs/session.tmp
touch -t 202604050000 project/charlie/temp/swap.tmp

# Recent files (< 30 days) — should NOT be touched
touch project/alice/reports/q3_current.log
touch project/bob/logs/today.log
touch project/charlie/temp/recent.tmp
```

2.2 Initial Directory Tree

The full file listing before running the script:

```
$ find /home/claude/project -type f | sort

/home/mehak/project/alice/reports/draft.tmp
/home/mehak/project/alice/reports/q1_summary.log
/home/mehak/project/alice/reports/q2_data.log
/home/mehak/project/alice/reports/q3_current.log
/home/mehak/project/bob/logs/server.log
/home/mehak/project/bob/logs/session.tmp
/home/mehak/project/bob/logs/today.log
/home/mehak/project/charlie/temp/cache.zip
/home/mehak/project/charlie/temp/recent.tmp
/home/mehak/project/charlie/temp/swap.tmp
/home/mehak/project/shared/archives/backup_old.zip
```

3. Script Design & Implementation

The script cleanup.sh was written in bash and organized into clearly separated phases. Below is the complete annotated script.

3.1 Variable Declarations & Setup

```
#!/bin/bash

PROJECT_DIR="/home/mehak/project"
BACKUP_BASE="/home/mehak/backup"
LOG_FILE="/home/mehak/cleanup.log"
REPORT_FILE="/home/mehak/report.txt"
BACKUP_MIN_FREE_MB=100

DATE_TAG=$(date +%Y-%m-%d)
BACKUP_DIR="${BACKUP_BASE}/backup_${DATE_TAG}"

FILES_MOVED=0
FILES_DELETED=0
SPACE_CLEARED_KB=0
PERM_ERRORS=0
```

All paths are stored in variables at the top so the script is easy to reconfigure. Counters track what happened during the run and feed into the final report.

3.2 Logging Function

```
log() {
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] $1" | tee -a "$LOG_FILE"
}
```

The log() function writes each message to cleanup.log with a full timestamp and also prints it to the terminal using tee. This makes monitoring easy without needing a separate tail -f.

3.3 Disk Space Guard (Open-ended: Out-of-Space Handling)

```
FREE_MB=$(df -m "$BACKUP_BASE" | awk 'NR==2 {print $4}')
log "Free space on backup volume: ${FREE_MB} MB"

if [ "$FREE_MB" -lt "$BACKUP_MIN_FREE_MB" ]; then
    log "ERROR: Not enough space. Need ${BACKUP_MIN_FREE_MB} MB, have ${FREE_MB}
MB."
    echo "CRITICAL: Backup aborted — insufficient disk space." >> "$REPORT_FILE"
    exit 2
fi
```

Before doing anything else, the script checks how much free space is available on the backup volume using df -m. If there is less than the configured threshold (100 MB), the script logs an error, writes a critical warning to the report, and exits with code 2. This prevents a half-finished backup from silently corrupting data.

3.4 Conflict Handling (Open-ended: Name Conflicts)

```
# For the backup folder itself:
if [ -d "$BACKUP_DIR" ]; then
    BACKUP_DIR="${BACKUP_DIR}_$(date +%H%M%S)"
    log "Backup folder existed. Using: $BACKUP_DIR"
fi
mkdir -p "$BACKUP_DIR"

# Per-file conflicts inside the backup:
if [ -e "$DEST_PATH" ]; then
    DEST_PATH="${DEST_PATH}.$(date +%s)"
    log "CONFLICT resolved: renaming to $(basename "$DEST_PATH")"
fi
```

Two levels of conflict resolution are implemented. At the folder level, if a backup folder for today already exists (e.g., the script is run twice in a day), a timestamp suffix (HHMMSS) is appended. At the file level, if a file with the same name already exists at the destination, a Unix timestamp is appended to the filename before moving. This ensures no data is ever silently overwritten.

3.5 Phase 1 — Delete Old .tmp Files

```
while IFS= read -r -d '' tmpfile; do
    SIZE_KB=$(du -k "$tmpfile" 2>/dev/null | cut -f1)
    if rm "$tmpfile" 2>/dev/null; then
        log "DELETED (tmp): $tmpfile  [${SIZE_KB}K]"
        SPACE_CLEARED_KB=$((SPACE_CLEARED_KB + SIZE_KB))
        FILES_DELETED=$((FILES_DELETED + 1))
    else
        log "PERM ERROR: Cannot delete $tmpfile"
        PERM_ERRORS=$((PERM_ERRORS + 1))
    fi
done < <(find "$PROJECT_DIR" -type f -name "*.tmp" -mtime +7 -print0)
```

3.6 Phase 2 — Move Old Files to Backup

```
while IFS= read -r -d '' oldfile; do
    RELATIVE="${oldfile#$PROJECT_DIR/}"
    DEST_DIR="${BACKUP_DIR}/$(dirname "$RELATIVE")"
    DEST_PATH="${BACKUP_DIR}/${RELATIVE}"

    mkdir -p "$DEST_DIR"

    if [ -e "$DEST_PATH" ]; then
        DEST_PATH="${DEST_PATH}.$(date +%s)"
    fi

    SIZE_KB=$(du -k "$oldfile" 2>/dev/null | cut -f1)

    if mv "$oldfile" "$DEST_PATH" 2>/dev/null; then
        log "MOVED: $oldfile -> $DEST_PATH  [${SIZE_KB}K]"
        SPACE_CLEARED_KB=$((SPACE_CLEARED_KB + SIZE_KB))
        FILES_MOVED=$((FILES_MOVED + 1))
    else
        log "PERM ERROR: Cannot move $oldfile"
        PERM_ERRORS=$((PERM_ERRORS + 1))
    fi
done < <(find "$PROJECT_DIR" -type f ! -name "*.tmp" -mtime +30 -print0)
```

Shell parameter expansion (${oldfile#$PROJECT_DIR/}) strips the project path prefix to reconstruct the relative path. mkdir -p then recreates the same subdirectory structure inside the backup folder. This preserves which user owned which file. .tmp files are excluded here with ! -name so they are not double-counted with Phase 1.

3.7 Phase 3 — Report Generation

```
{
    echo "========================================"
    echo "  Backup & Cleanup Report"
    echo "  Generated : $(date '+%Y-%m-%d %H:%M:%S')"
    echo "  Backup dir: $BACKUP_DIR"
    echo "========================================"
    echo "SUMMARY"
    echo "  Files moved to backup : $FILES_MOVED"
    echo "  Temp files deleted    : $FILES_DELETED"
    printf "  Total space cleared   : %s MB\n" \
        "$(echo "scale=2; $SPACE_CLEARED_KB/1024" | bc)"
    echo "  Permission errors     : $PERM_ERRORS"
    echo "FILES MOVED"
    find "$BACKUP_DIR" -type f | while read -r f; do echo "  $f"; done
    echo "REMAINING IN PROJECT"
    find "$PROJECT_DIR" -type f | while read -r f; do echo "  $f"; done
} > "$REPORT_FILE"
```

The entire report is built inside a { } block and redirected as a single operation to report.txt. bc is used for floating-point division when converting kilobytes to megabytes, since bash cannot handle decimals natively.

4. Script Execution & Terminal Output

The script was made executable and run with the following commands:

```
$ chmod +x cleanup.sh
$ bash cleanup.sh
```

4.1 Terminal Output (cleanup.log contents)

The script printed the following timestamped log to the terminal and simultaneously wrote it to cleanup.log:

```
[2026-04-16 07:11:43] ====== Cleanup session started ======
[2026-04-16 07:11:43] Free space on backup volume: 10048 MB
[2026-04-16 07:11:44] Backup destination: /home/mehak/backup/backup_2026-04-16
[2026-04-16 07:11:44] --- Phase 1: Deleting .tmp files older than 7 days ---
[2026-04-16 07:11:44] DELETED (tmp): /home/mehak/project/alice/reports/draft.tmp
[0K]
[2026-04-16 07:11:44] DELETED (tmp): /home/mehak/project/charlie/temp/swap.tmp  [0K]
[2026-04-16 07:11:44] DELETED (tmp): /home/mehak/project/bob/logs/session.tmp  [0K]
[2026-04-16 07:11:44] --- Phase 2: Moving files older than 30 days to backup ---
[2026-04-16 07:11:44] MOVED: /home/mehak/project/shared/archives/backup_old.zip
                           ->  /home/mehak/backup/backup_2026-04-
16/shared/archives/backup_old.zip  [500K]
[2026-04-16 07:11:44] MOVED: /home/mehak/project/alice/reports/q2_data.log
                           ->  /home/mehak/backup/backup_2026-04-
16/alice/reports/q2_data.log  [1K]
[2026-04-16 07:11:44] MOVED: /home/mehak/project/alice/reports/q1_summary.log
                           ->  /home/mehak/backup/backup_2026-04-
16/alice/reports/q1_summary.log  [1K]
[2026-04-16 07:11:44] MOVED: /home/mehak/project/charlie/temp/cache.zip
                           ->  /home/mehak/backup/backup_2026-04-
16/charlie/temp/cache.zip  [200K]
[2026-04-16 07:11:44] MOVED: /home/mehak/project/bob/logs/server.log
                           ->  /home/mehak/backup/backup_2026-04-
16/bob/logs/server.log  [1K]
[2026-04-16 07:11:44] --- Phase 3: Generating report ---
[2026-04-16 07:11:44] Report written to /home/mehak/report.txt
[2026-04-16 07:11:44] ====== Cleanup session complete ======
[2026-04-16 07:11:44]   Moved=5 | Deleted=3 | Space=703K | Errors=0
```

5. Generated Outputs

5.1 report.txt

The following file was generated at /home/mehak/report.txt after the run:

```
========================================
  Backup & Cleanup Report
  Generated : 2026-04-16 07:11:44
  Backup dir: /home/mehak/backup/backup_2026-04-16
========================================

SUMMARY
-------
  Files moved to backup : 5
  Temp files deleted    : 3
  Total space cleared   : .68 MB
  Permission errors     : 0

FILES MOVED
-----------
  /home/mehak/backup/backup_2026-04-16/shared/archives/backup_old.zip
  /home/mehak/backup/backup_2026-04-16/alice/reports/q2_data.log
  /home/mehak/backup/backup_2026-04-16/alice/reports/q1_summary.log
  /home/mehak/backup/backup_2026-04-16/charlie/temp/cache.zip
  /home/mehak/backup/backup_2026-04-16/bob/logs/server.log

REMAINING IN PROJECT (recent files kept)
----------------------------------------
  /home/mehak/project/alice/reports/q3_current.log
  /home/mehak/project/charlie/temp/recent.tmp
  /home/mehak/project/bob/logs/today.log

========================================
```

5.2 Final Directory State

After the script completed, the backup folder preserved the original subdirectory structure:

```
$ find /home/mehak/backup -type f | sort

/home/mehak/backup/backup_2026-04-16/alice/reports/q1_summary.log
/home/mehak/backup/backup_2026-04-16/alice/reports/q2_data.log
/home/mehak/backup/backup_2026-04-16/bob/logs/server.log
/home/mehak/backup/backup_2026-04-16/charlie/temp/cache.zip
/home/mehak/backup/backup_2026-04-16/shared/archives/backup_old.zip
```

Only recent files (less than 30 days old) remained in the project directory:

```
$ find /home/mehak/project -type f | sort

/home/mehak/project/alice/reports/q3_current.log
/home/mehak/project/bob/logs/today.log
/home/mehak/project/charlie/temp/recent.tmp
```